That's it. Our app is live on the Swarm cluster. Let's visit http://any of nodes ips/welcome—you should see the hit counter. Also navigate to http://any of nodes ips/users to see that the User scaffold is working too.

If you're wondering how our database exists, that's thanks to our database service's Postgres image, which automatically creates the default database, as specified by the POSTGRES_DB environment variable we set.[7] The database has also been migrated thanks to our handy database-migrator service we added on page 163.

Visualizing Containers

As we've discussed, Swarm's orchestrator schedules containers (or more strictly, *tasks* to run containers) on whichever nodes it sees fit. That means we don't know where a container for a service will end up running.

For educational purposes, we're going to use a new "visualizer" tool that provides a web interface for seeing the nodes in our clusters, and the containers running on them; Docker provides a handy image for this. You wouldn't typically run the visualizer in production, but it will give us a feel for how containers are scheduled across the cluster.

Let's add a visualizer service to our docker-stack.yml file:

```
Line 1  visualizer:
  2       image: dockersamples/visualizer:stable
  3       ports:
  4         - "8080:8080"
  5       volumes:
  6         - "/var/run/docker.sock:/var/run/docker.sock"
  7       deploy:
  8         placement:
  9           constraints: [node.role == manager]
```

In order to view the web interface, we need to expose a port publicly. Internally, the visualizer runs on port 8080. Since nothing else in our cluster runs on this port, we've chosen to use it as the public port we expose (lines 3–4).

The visualizer relies on having access to a file called /var/run/docker.sock, which is available on the Dockerhost. We mount it into the container as a volume (line 6).

7. https://hub.docker.com/_/postgres/